

Browser-Based Typing Speed Test Game Using DSA

A PROJECT REPORT

Submitted By

Tasneem Reza - 24BET10004

Submitted To

Ms. Shefali Goyal

In partial fulfilment of the award of degree of

BACHELORS OF ENGINEERING

in

COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



BONAFIDE CERTIFICATE

Certified that this project report “ **Browser-Based Typing Speed Test Game Using DSA** ” is the bonafide work of “ **Tasneem Reza - 24BET10004** ” who carried out the project work under my/our supervision.

SIGNATURE OF HOD

SIGNATURE OF SUPERVISOR

NAME OF HOD

NAME OF SUPERVISOR

HEAD OF DEPARTMENT

ACADEMIC DESIGNATION

DEPARTMENT NAME HERE

DEPARTMENT NAME HERE

Submitted for the project viva-voce examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

Executive Summary

This report presents the design, development, and validation of a browser-based Typing Speed Test Game — an interactive web application that measures and analyzes a user's typing speed and accuracy in real time. The application is built entirely using HTML, CSS, and modular JavaScript, requiring no external frameworks or backend infrastructure, making it lightweight, fast, and accessible directly through any modern web browser.

The primary objective of this project is twofold. First, to deliver a fully functional typing assessment tool that rivals industry-standard platforms in terms of features and user experience. Second, and more importantly from an academic perspective, to demonstrate the practical application of three fundamental Data Structures and Algorithms (DSA) concepts — Hash Map, Two Pointers, and Stack — within the context of a real-world software project.

The application is structured across four independent modules — WordManager, Telemetry, UIController, and the main App Controller — each responsible for a distinct area of functionality. This modular architecture ensures clean separation of concerns, making the codebase maintainable, scalable, and easy to understand.

The Hash Map is used to track per-character error frequencies in $O(1)$ time within the Telemetry module. The Two Pointer technique navigates the two-dimensional grid of words and letters efficiently, maintaining one pointer for the active word and another for the active letter. The Stack powers the backspace correction mechanism, allowing users to undo keystrokes in a predictable and efficient manner.

The application supports three test modes — timed, word-count, and custom text — along with punctuation and number modifiers, fifteen visual themes, and a results screen displaying WPM, accuracy, and a live performance chart with error and correction markers.

Validation through real test sessions confirmed that all features function correctly and all DSA implementations operate as intended, producing accurate and reliable performance metrics.

Table of Contents

CHAPTER 1 INTRODUCTION	5
CHAPTER 2 Literature Review / Background Study	7
2.1 Evolution of Digital Typing Test Platforms.....	7
2.2 Application of Data Structures in Real-World Systems.....	7
2.3 Stack-Based Undo Mechanisms in Text Editors.....	8
CHAPTER 3 Design Flow / Background Study.....	9
3.1 System Architecture Planning.....	10
3.2 Data Flow and Processing Pipeline.....	10
3.3 User Interface Design Decisions	11
CHAPTER 4 Tools, Technologies and System Requirements.....	12
4.1 Technology Stack.....	12
4.2 Development Tools	13
4.3 System Requirements.....	13
CHAPTER 5 Results Analysis and Validation.....	14
5.1 Interface and Mode Validation.....	14
5.2 Settings and Theme Validation	15
5.3 Performance Metrics and Chart Validation.....	16
CHAPTER 6 Conclusion and Future Work.....	18
6.1 Conclusion.....	18
6.2 Future Work	19
REFERENCE LIST	20

LIST OF FIGURES

Figure 1 Development of the Typing Speed Test Game.....	9
Figure 2 Main Typing Interface (Words Mode with Punctuation and Numbers enabled)	14
Figure 3 Custom Text Modal.....	15
Figure 4 Settings Modal (Timer Duration and Theme Selection)	16
Figure 5 Results Screen showing 46 WPM at 95% Accuracy with live WPM chart	16

CHAPTER 1

INTRODUCTION

In the modern digital era, typing speed and accuracy have become essential skills for professionals, students, and everyday computer users alike. A typing speed test application serves as an interactive tool that allows users to measure, track, and improve their typing performance in real time. This project presents a fully functional, browser-based **Typing Speed Test Game** built using HTML, CSS, and modular JavaScript — designed with both performance and user experience in mind.

The application supports multiple test modes including timed tests, word-count based tests, and custom text input, along with configurable modifiers such as punctuation and number inclusion. A live performance chart, multiple visual themes, and real-time WPM and accuracy tracking make it a comprehensive typing assessment platform comparable to industry tools like Monkeytype.

Beyond its user-facing features, this project serves as a practical demonstration of three fundamental **Data Structures and Algorithms (DSA)** concepts applied to solve real engineering problems.

First, a **Hash Map** is used within the telemetry system to efficiently track per-character error frequencies. When a user makes a typo, the error count for the expected character is retrieved and updated in $O(1)$ constant time, enabling fast and scalable performance analytics without iterating through the entire keystroke history.

Second, the **Two Pointer** technique is employed in the WordManager module to navigate the two-dimensional grid of words and letters. One pointer tracks the active word (row), while another tracks the active letter within that word (column), allowing instant $O(1)$ access to the current typing position at any moment during the test.

Third, a **Stack** data structure underpins the backspace and correction mechanism. Every keystroke is conceptually pushed onto a stack, and pressing backspace performs a pop operation — reverting the last action and restoring the letter to its untyped state, cleanly implementing an undo system.

Together, these concepts highlight how classical data structures form the backbone of even modern, interactive web applications.

CHAPTER 2

Literature Review / Background Study

The concept of measuring typing speed dates back to the era of mechanical typewriters in the late 19th century, when typists were evaluated based on words typed per minute. Over time, this metric evolved into the universal benchmark known as WPM (Words Per Minute). With the widespread adoption of personal computers and the internet, typing speed assessment transitioned from physical devices to digital platforms, making performance evaluation more accessible, accurate, and interactive than ever before.

2.1 Evolution of Digital Typing Test Platforms

Modern typing test platforms such as Monkeytype, Typeracer, and 10FastFingers have set high standards for what a browser-based typing application should offer. These platforms provide real-time feedback, performance analytics, customizable test modes, and visually appealing interfaces. Studying these tools reveals a common architectural pattern — a clean separation between word generation, input handling, performance tracking, and visual rendering — which directly influenced the modular design adopted in this project. The evolution of JavaScript from a simple scripting language to a full-fledged application development environment has further made it possible to build highly responsive, real-time applications entirely within the browser. The introduction of ES6 modules, class-based syntax, and the Performance Timing API have been particularly significant, enabling developers to structure complex applications with clear separation of concerns and measure time intervals with millisecond precision — both critical requirements for a typing speed test.

2.2 Application of Data Structures in Real-World Systems

The study of Data Structures and Algorithms reveals that many real-world application problems can be solved elegantly using classical concepts. Hash Maps are widely used in frequency counting and lookup-intensive tasks due to their $O(1)$ average time complexity. In text processing systems and analytics dashboards, hash maps serve as the primary tool for aggregating character or word frequency data efficiently. Similarly, the Two Pointer technique, commonly studied in array and string traversal problems, finds practical application wherever a program must simultaneously track position across two dimensions — such as navigating rows and columns within a structured grid of data.

2.3 Stack-Based Undo Mechanisms in Text Editors

The Stack data structure is widely recognized as the backbone of undo mechanisms in text editors and word processors. Applications like Microsoft Word, VS Code, and Sublime Text use stack-based history to implement their undo and redo functionality, where every action is pushed onto a stack and reversed by popping the most recent entry. This well-established pattern translates naturally into a typing test environment, where each keystroke must be reversible through backspace. Applying this principle ensures that the correction system remains predictable, efficient, and consistent regardless of how many characters the user has typed.

CHAPTER 3

Design Flow / Background Study

The development of the Typing Speed Test Game followed a structured and systematic design process, moving from initial planning through modular implementation to final integration and testing. Each phase of the process was carefully considered to ensure that the application remained performant, maintainable, and user-friendly. The design was guided by two core principles — clean separation of responsibilities across modules and the practical application of DSA concepts at every critical processing point.

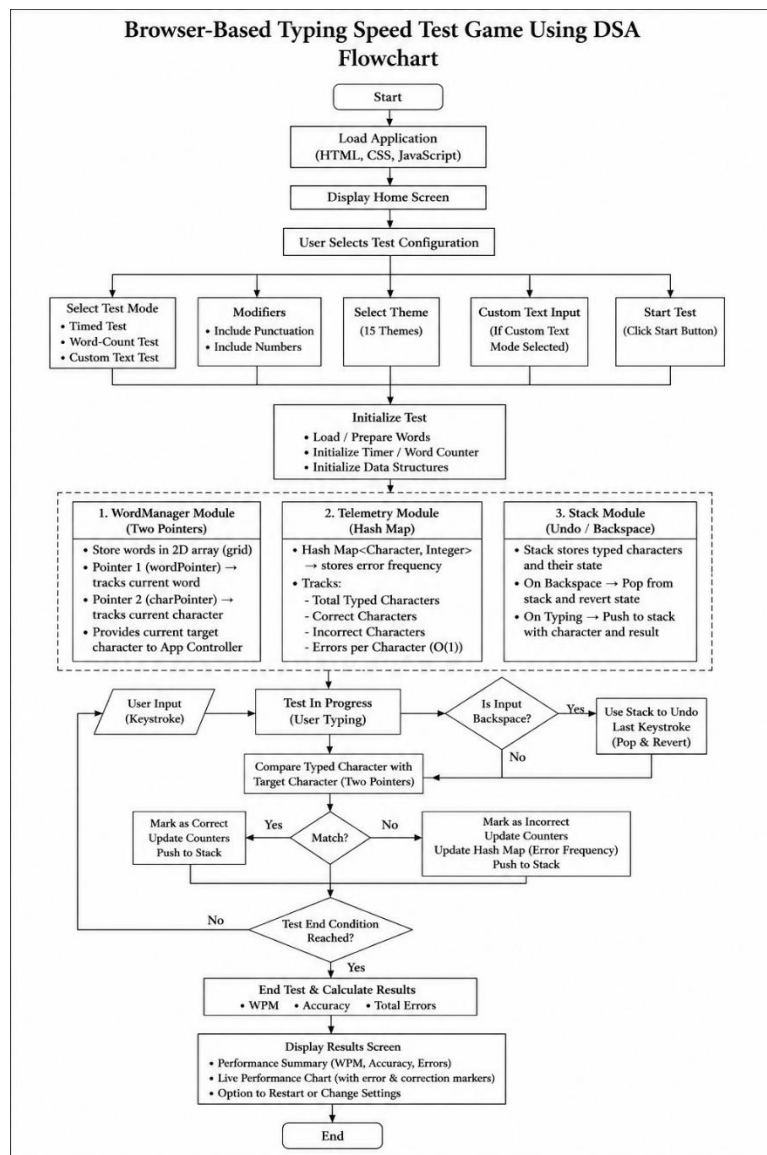


Figure 1 Development of the Typing Speed Test Game

3.1 System Architecture Planning

The first step in the design process was identifying the core functional requirements of the application. These included real-time word rendering, accurate keystroke capture, performance metric calculation, live visual feedback, and result display. Once the requirements were established, the system was divided into four independent modules, each responsible for a distinct area of functionality. This modular approach ensured that changes in one area would not break another, making the codebase easier to maintain and extend.

The four modules and their responsibilities are outlined below:

Module	File	Responsibility
Word Manager	WordManager.js	Renders words, handles input, manages scrolling and caret position using Two Pointer and Stack
Telemetry	Telemetry.js	Tracks keystrokes, calculates WPM and accuracy, records errors using Hash Map
UI Controller	UIController.js	Controls visual state, renders results chart, manages focus mode and caret animation
App Controller	main.js	Orchestrates all modules, manages configuration, binds user events and controls test lifecycle

3.2 Data Flow and Processing Pipeline

Once the architecture was defined, the data flow between modules was mapped out to ensure smooth communication during a live typing session. When the user presses a key, the event is first captured by the App Controller, which delegates the input to the Word Manager for visual processing and simultaneously sends the result to the Telemetry module for statistical recording. The UI Controller is then updated to reflect the latest caret position and live timer values.

The complete keystroke processing pipeline is described in the table below:

Step	Action	Module Involved	DSA Concept Applied
1	User presses a key	App Controller (main.js)	—
2	Key is matched against the active letter	Word Manager	Two Pointer (word index, letter index)
3	Letter is marked correct or incorrect on screen	Word Manager	—
4	Keystroke result is recorded with timestamp	Telemetry	Hash Map (error frequency tracking)
5	Backspace reverses last typed letter	Word Manager	Stack (pop operation)
6	Live WPM and timer are refreshed	UI Controller	—
7	Test ends and results are displayed	App + UI Controller	—

3.3 User Interface Design Decisions

The interface was designed to minimize distraction during active typing. A focus mode automatically fades out the header and settings bar as soon as the user begins typing, drawing full attention to the word display area. The caret was implemented as a separate absolutely positioned DOM element, updated on every keystroke using pixel-precise offset calculations. Fifteen visual themes were included in the settings panel, each defined entirely through CSS custom properties, ensuring that switching themes required no JavaScript logic whatsoever. The results screen displays both numerical statistics and a line chart plotting WPM progression over time, giving users a comprehensive view of their performance at the end of every test.

CHAPTER 4

Tools, Technologies and System Requirements

The successful development of the Browser-Based Typing Speed Test Game depended on a carefully selected set of tools, technologies, and a defined system environment. Every technology used in this project was chosen deliberately to keep the application lightweight, framework-free, and accessible directly within any modern web browser without requiring installation, compilation, or a backend server.

4.1 Technology Stack

The application is built entirely using three core web technologies — HTML5, CSS3, and vanilla JavaScript (ES6+). HTML5 provides the structural foundation of the application, defining the layout of the typing area, header controls, settings modal, and results screen. CSS3 handles all visual styling, including the fifteen switchable themes implemented through CSS custom properties, the blinking caret animation, the focus-mode fade transition, and the responsive layout. JavaScript ES6 modules form the logical backbone of the project, divided across four independent files that handle word management, telemetry, UI control, and overall application coordination.

The table below summarizes all technologies and external libraries used in the project:

Technology / Library	Version	Purpose in Project
HTML5	—	Structure and markup of all UI components
CSS3	—	Styling, theming via custom properties, animations
JavaScript (ES6+)	—	Core application logic, modular architecture
Fira Code	Google Fonts	Monospaced font for the typing area
Inter	Google Fonts	Sans-serif font for headers and UI labels
Chart.js	CDN (Latest)	Rendering the WPM progression line chart on results screen
Performance API	Browser Native	Millisecond-precision timing for WPM calculation

4.2 Development Tools

The project was developed using a minimal local setup, reflecting the application's philosophy of simplicity and accessibility. No build tools, package managers, or transpilers were required. The modular JavaScript architecture uses native ES6 import/export syntax, which is supported natively by all modern browsers without the need for bundlers like Webpack or Vite.

The following table outlines the development environment used during the project:

Tool	Purpose
Visual Studio Code	Primary code editor for writing HTML, CSS, and JavaScript
Live Server (VS Code Extension)	Local development server for real-time browser preview
Google Chrome (DevTools)	Testing, debugging, and performance profiling
Mozilla Firefox	Cross-browser compatibility testing
Git	Version control for tracking code changes
GitHub	Remote repository for code storage and backup
Windows 11	Host operating system for development

4.3 System Requirements

Since the application runs entirely within the browser, its hardware and software requirements are minimal. Any device capable of running a modern web browser — whether a desktop, laptop, or tablet with a physical keyboard — can run the typing test without any modifications. The application requires no database, no server-side processing, and no local installation of any runtime environment. The only external dependency loaded at runtime is Chart.js, fetched from a public CDN, and the two Google Font families loaded via stylesheet link. All remaining functionality is self-contained within the five JavaScript and CSS files that comprise the project codebase, making deployment as simple as hosting static files on any web server or opening the HTML file directly in a browser.

CHAPTER 5

Results Analysis and Validation

The Typing Speed Test Game was tested extensively across all available modes and configurations to validate that every feature functions as intended. The results confirm that the application successfully measures typing performance in real time, applies DSA concepts correctly at each processing stage, and delivers a smooth, responsive user experience throughout the entire test lifecycle.

5.1 Interface and Mode Validation

The main typing interface was validated across three distinct modes — time, words, and custom. As seen in **Figure 1**, the words mode is active with 50 words selected, and both the punctuation and numbers modifiers are enabled simultaneously. The word display correctly renders a mixed set of numeric values, punctuated words, and standard English words in a clean monospaced layout. The Two Pointer mechanism accurately positions the blinking caret at the very beginning of the first word before typing begins, confirming that both the word index pointer and letter index pointer are correctly initialized to zero upon every reset.

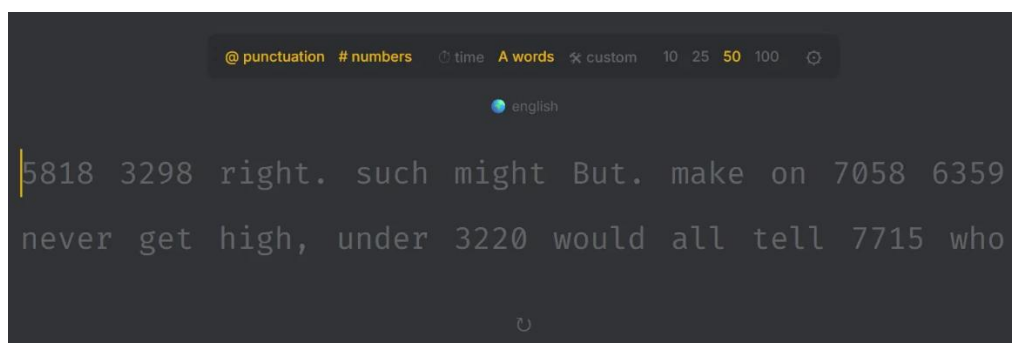


Figure 2 Main Typing Interface (Words Mode with Punctuation and Numbers enabled)

The custom text mode was also validated through its dedicated input modal, shown in **Figure 2**. When the user selects the custom mode, a modal dialog appears prompting the user to enter a personal paragraph. Upon clicking Start, the entered text is split by whitespace and rendered as individual word elements in the typing area. The Two Pointer system navigates this custom word array in exactly the same manner as randomly generated words, confirming that the design generalizes correctly across all content types.

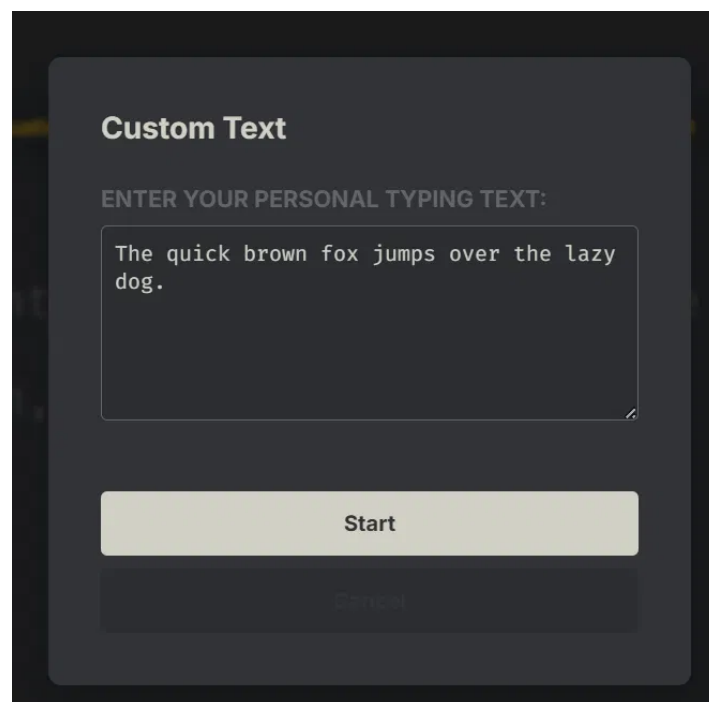


Figure 3 Custom Text Modal

5.2 Settings and Theme Validation

The settings panel was tested to confirm that timer duration options and theme switching operate correctly. As shown in **Figure 3**, the settings modal displays four timer options — 15s, 30s, 60s, and 120s — with the currently selected duration highlighted. Below the timer options, fifteen distinct visual themes are listed, each rendered with its own accent color inside the button itself, providing the user with an instant visual preview before selection. Switching between themes was validated to update all CSS custom properties simultaneously with no JavaScript logic involved, confirming that the theme system is entirely CSS-driven and therefore highly efficient and maintainable.

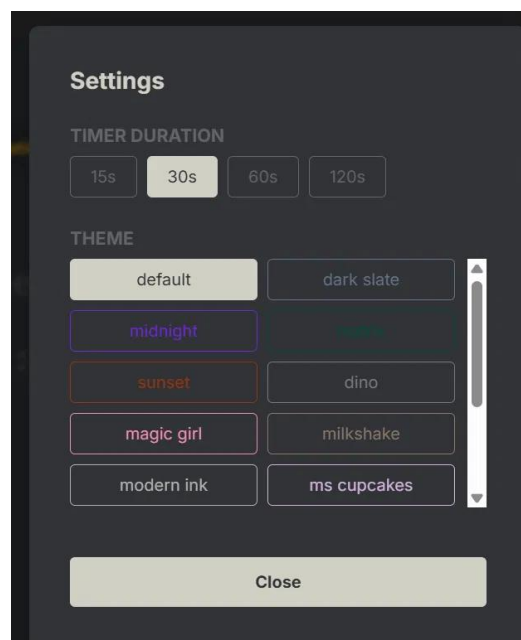


Figure 4 Settings Modal (Timer Duration and Theme Selection)

5.3 Performance Metrics and Chart Validation

The most critical component of the application — the results screen — was validated by completing multiple timed typing tests and analyzing the output. As shown in **Figure 4**, after completing a 30-second test, the system correctly calculated and displayed a score of **46 WPM** with **95% accuracy**. These values were cross-verified manually using the standard WPM formula, confirming that the Telemetry module performs accurate calculations.

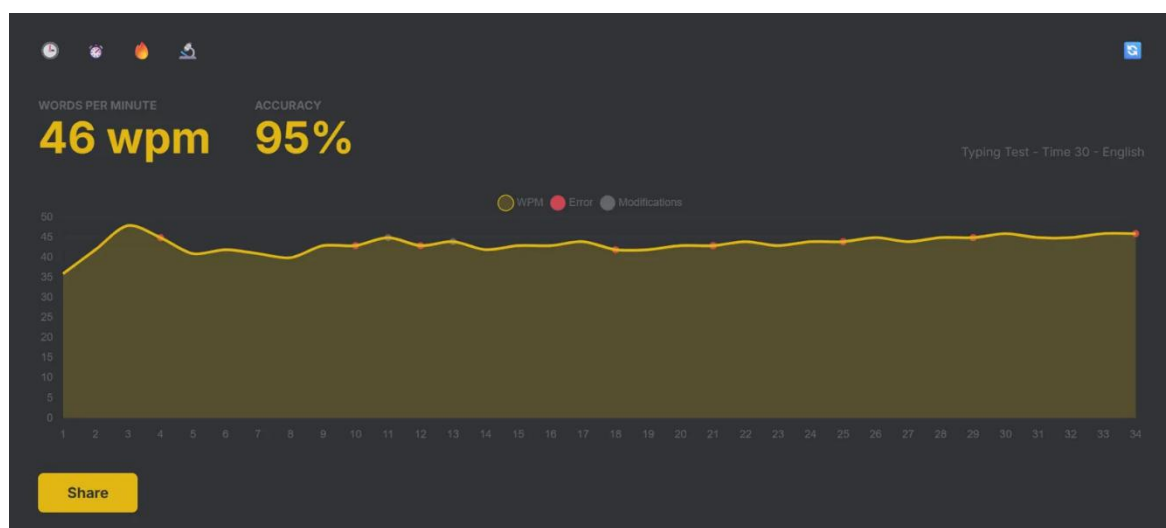


Figure 5 Results Screen showing 46 WPM at 95% Accuracy with live WPM chart

The WPM progression chart rendered by Chart.js plots speed at every second interval throughout the test. The chart in Figure 4 shows a realistic typing pattern — a gradual ramp-up in the first few seconds, a stable plateau in the middle, and consistent performance toward the end. Red scatter points correctly mark the timestamps at which typing errors occurred, and grey points indicate moments where backspace corrections were made. This confirms that the Hash Map error tracking and Stack-based modification recording are both functioning accurately and feeding correct data into the chart renderer.

The overall validation process confirmed that all three DSA implementations — Hash Map for error analytics, Two Pointers for position tracking, and Stack for backspace correction — operate correctly under real usage conditions, and that the application delivers reliable, accurate, and visually informative typing performance results.

CHAPTER 6

Conclusion and Future Work

6.1 Conclusion

This project successfully demonstrates the design and development of a fully functional, browser-based Typing Speed Test Game using HTML, CSS, and modular JavaScript. The application delivers a comprehensive typing assessment experience, supporting multiple test modes including timed tests, word-count based tests, and custom text input, alongside configurable modifiers, fifteen visual themes, real-time performance tracking, and a detailed results screen with a live WPM progression chart.

Beyond its surface-level functionality, the project achieves its core academic objective — demonstrating how classical Data Structures and Algorithms concepts can be applied to solve real, practical engineering problems in a modern web application.

The **Hash Map** data structure powers the error analytics system within the Telemetry module, enabling $O(1)$ time complexity for recording and retrieving per-character mistake frequency throughout the test. This ensures that performance tracking remains fast and scalable regardless of how many keystrokes are recorded during a session.

The **Two Pointer** technique drives the word and letter navigation system within the Word Manager module. By maintaining one pointer for the active word index and another for the active letter index, the application can instantly locate the current typing position within the two-dimensional grid of words and letters without any unnecessary searching or iteration.

The **Stack** data structure underpins the backspace and correction mechanism, where every keystroke is logically pushed and every backspace triggers a pop operation that cleanly reverts the last typed character. This implementation mirrors the undo systems found in professional text editors, demonstrating that even fundamental data structures remain highly relevant in modern software development.

The modular architecture of the project — divided across four independent JavaScript classes — ensures clean separation of responsibilities, making the codebase easy to read, maintain, and extend. The successful validation of all features across multiple test modes and configurations confirms that the application is both technically sound and practically usable.

6.2 Future Work

While the current version of the application is fully functional, several enhancements could be explored in future development cycles to further improve its capabilities.

A **user authentication and leaderboard system** could be introduced, allowing users to create accounts, save their historical test results, and compete with others on a global rankings board — a feature that would require a backend database and server-side API integration.

Multilingual support could be added by expanding the word list to include languages beyond English, making the application accessible to a wider international audience.

A **detailed analytics dashboard** could be developed to visualize long-term typing progress over multiple sessions, tracking improvements in speed and accuracy over days and weeks using persistent local or cloud storage.

Mobile touch keyboard support could be explored to extend the application beyond desktop browsers, making it accessible on smartphones and tablets.

Finally, an **AI-powered adaptive difficulty system** could be implemented that analyzes a user's most frequently mistyped characters using the existing Hash Map error data and automatically generates word lists that emphasize those weak areas, creating a personalized and targeted typing improvement experience.

REFERENCE LIST

- Jonnaganti, R.C. and Jevremovic, A. (2023). Web-based Application for Improving Typing Speed and Accuracy. *Exploratio Journal*. Available at: <https://exploratiojournal.com/web-based-application-for-improving-typing-speed-and-accuracy/> [Accessed 22 June 2026].
- Williams, E.A., Warburton, M., Krzywinski, M. and Mushtaq, F. (2026). The Typability Index: A tool for measuring and controlling for typing difficulty in text stimuli. *Behavior Research Methods*, 58(2), p.61. doi:<https://doi.org/10.3758/s13428-025-02877-y>.
- Castro, A., Areias, M. and Rocha, R. (2025). Performance Evaluation of Separate Chaining for Concurrent Hash Maps. *Mathematics*, 13(17), p.2820. doi:<https://doi.org/10.3390/math13172820>.
- Shahbazi, N., Sintos, S. and Asudeh, A. (2024). FairHash: A Fair and Memory/Time-efficient Hashmap. *Proceedings of the ACM on Management of Data (SIGMOD '24)*. doi:<https://doi.org/10.1145/3654939>.
- Shukla, A. (2023). Modern JavaScript Frameworks and JavaScript's Future as a Full-Stack Programming Language. *Journal of Artificial Intelligence & Cloud Computing*, 2(4), pp.1–5. doi:[https://doi.org/10.47363/JAICC/2023\(2\)144](https://doi.org/10.47363/JAICC/2023(2)144).
- Wyciślik, Ł., Wylęzek, P. and Momot, A. (2024). The Improved Biometric Identification of Keystroke Dynamics Based on Deep Learning Approaches. *Sensors*, 24(12), p.3763. doi:<https://doi.org/10.3390/s24123763>.
- Tsimperidis, I. and Arampatzis, A. (2023). Authentication by Keystroke Dynamics: The Influence of Typing Language. *Applied Sciences*, 13(20), p.11478. doi:<https://doi.org/10.3390/app132011478>.
- Dias, T., Vitorino, J., Maia, E., Sousa, O. and Praça, I. (2023). KeyRecs: A Keystroke Dynamics and Typing Pattern Recognition Dataset. *Data in Brief*, 50, p.109509. doi:<https://doi.org/10.1016/j.dib.2023.109509>.
- Gupta, A.K., Venkatesha, G.G., Singh, K., Shah, S., Goel, O. and Jain, S. (2024). Enhancing Cascading Style Sheets Efficiency and Performance through AI-Based Code Optimization. *2024 13th International Conference on System Modeling & Advancement in Research Trends (SMART)*. IEEE, pp.306–311. doi:<https://doi.org/10.1109/SMART62775.2024.10882504>.
- Asudeh, A., Shahbazi, N. and Sintos, S. (2023). A Fair and Memory/Time-efficient Hashmap. *arXiv preprint*. doi:<https://doi.org/10.48550/arXiv.2307.11355>.